

Assignment 2: Coding Basics

Esther Yang

OVERVIEW

This exercise accompanies the lessons/labs in Environmental Data Analytics on coding basics.

Directions

1. Rename this file `<FirstLast>_A02_CodingBasics.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change “Student Name” on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure to **answer the questions** in this assignment document.
5. When you have completed the assignment, **Knit** the text and code into a single PDF file.
6. After Knitting, submit the completed exercise (PDF file) to Canvas.

Basics, Part 1

1. Generate a sequence of numbers from one to 55, increasing by fives. Assign this sequence a name.
2. Compute the mean and median of this sequence.
3. Ask R to determine whether the mean is greater than the median.
4. Insert comments in your code to describe what you are doing.

```
#1. Generate a sequence of numbers from one to 55, increasing by fives. Name this sequence seq_by_five.
seq_by_five <- seq(1, 55, by = 5)
```

```
#2. Compute the mean and median of this sequence.
```

```
mean_seq <- mean(seq_by_five)
```

```
median_seq <- median(seq_by_five)
```

```
#3. Ask R to determine whether the mean is greater than the median.
```

```
mean_greater <- mean_seq > median_seq
```

```
#4. Print the results.
```

```
print(paste("Mean:", mean_seq))
```

```
## [1] "Mean: 26"
```

```
print(paste("Median:", median_seq))
```

```
## [1] "Median: 26"
```

```
print(paste("Is mean greater than the median?", mean_greater))
```

```
## [1] "Is mean greater than the median? FALSE"
```

Basics, Part 2

5. Create three vectors, each with four components, consisting of (a) student names, (b) test scores, and (c) whether they are on scholarship or not (TRUE or FALSE).
6. Label each vector with a comment on what type of vector it is.
7. Combine each of the vectors into a data frame. Assign the data frame an informative name.
8. Label the columns of your data frame with informative titles.

```
#5a. Character vector of student names.
student_names <- c("Esther", "Emma", "Graves", "Joe")

#5b. Numeric vector of test scores.
test_scores <- c(99, 98, 97, 96)

#5c. Logical vector for scholarship status.
scholarship_status <- c(TRUE, FALSE, TRUE, FALSE)

#7. Combine each of the vectors into a data frame.
students_data <- data.frame(
  #8. Label the columns of your data frame with informative titles.
  Name = student_names,
  Score = test_scores,
  Scholarship = scholarship_status
)

#8. Show the data frame.
print(students_data)
```

```
##      Name Score Scholarship
## 1 Esther    99         TRUE
## 2  Emma    98         FALSE
## 3 Graves    97         TRUE
## 4   Joe    96         FALSE
```

9. QUESTION: How is this data frame different from a matrix?

Answer: The difference between this data frame and a matrix is that this data frame can contain columns of different data types which are character, numeric, and logical, while a matrix must contain elements of only one data type.

10. Create a function with one input. In this function, use `if...else` to evaluate the value of the input: if it is greater than 50, print the word “Pass”; otherwise print the word “Fail”.
11. Create a second function that does the exact same thing as the previous one but uses `ifelse()` instead of `if...else`.

12. Run both functions using the value 52.5 as the input
13. Run both functions using the **vector** of student test scores you created as the input. (Only one will work properly...)

```
#10. Create a function using if...else.
evaluate_score_if <- function(score) {
  if(score > 50) {
    return("Pass")
  } else {
    return("Fail")
  }
}

#11. Create a function using ifelse().
evaluate_score_ifelse <- function(score) {
  ifelse(score > 50, "Pass", "Fail")
}

#12a. Run the first function with the value 52.5.
result_12a <- evaluate_score_if(52.5)
print(result_12a)
```

```
## [1] "Pass"
```

```
#12b. Run the second function with the value 52.5.
result_12b <- evaluate_score_ifelse(52.5)
print(result_12b)
```

```
## [1] "Pass"
```

```
#13a. Run the first function with the vector of test scores.
# result_13a <- evaluate_score_if(test_scores)
# print(result_13a)

#13b. Run the second function with the vector of test scores.
result_13b <- evaluate_score_ifelse(test_scores)
print(result_13b)
```

```
## [1] "Pass" "Pass" "Pass" "Pass"
```

14. QUESTION: Which option of `if...else` vs. `ifelse` worked? Why? (Hint: search the web for “R vectorization”)

Answer: The `ifelse()` function worked when applied to the vector of test scores, while the `if...else` statement did not. This difference occurs because `ifelse()` is a vectorized function which can handle vector inputs, while the base `if...else` statement is not vectorized and can only handle a single logical condition at a time.

NOTE Before knitting, you'll need to comment out the call to the function in Q13 that does not work. (A document can't knit if the code it contains causes an error!)